

Automatic Differentiation

Jakub Vokoun

Abstract—This paper provides an explanation of Automatic Differentiation (AD) concept for university students at near-bachelor level. Includes comparison with other differentiation methods, very simple AD implementation example and more realistic usage in python library.

Index Terms—autodiff, python, computational mathematics

I. INTRODUCTION

Derivative is an indispensable tool in almost any field of mathematics. Its useful property lies in revealing the rate of change of any function. The process of differentiation is very simple, therefore presumably easily encoded into computer program. From the birth of machine computing various methods were discovered, all with their respective drawbacks. We will explore these methods later. It is necessary to understand the foundational challenges and inner-workings of differentiation first.

Derivative may be defined as a limit, such as

$$L = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h} \quad (1)$$

where $f(x)$ is differentiable on an open interval containing point a , and if the limit L exists. In principle, the derivative of any function (if exists) may be computed using this definition. However, since was discovered different approach. When a few simple functions have known derivatives, using them with a set of rules for obtaining derivatives of more complicated functions results in a different, simpler method. The process of finding a derivative is called differentiation.

For example, if we know the derivative of a polynomial

$$(x^a)' = ax^{a-1} \quad (2)$$

where a is an arbitrary integer, and a sum rule

$$(\alpha f + \beta g)' = \alpha f' + \beta g' \quad (3)$$

for any function f, g , and all real numbers α, β , we may obtain derivative for this function:

$$f(x) = 3x^2 + 5x \quad (4)$$

Using both the rules its apparent the result will be:

$$f(x)' = 6x + 5 \quad (5)$$

This simple example demonstrated, that with as little as one known derivative and one rule, we are able to differentiate a whole class of functions, polynomials.

If we add to our set of known derivatives and rules, we are quickly able to differentiate any known function. With large enough set it is only matter of mechanically applying rules and known derivatives in the correct order. Something a computer is supposedly very good at, or at least less error-prone than most humans.

II. BACKGROUND

TODO: proč se na to hodí použít počítač, fundamentální problém (zdrojový kód přesně nekopíruje matematické funkce), možná zase motivace

III. ALTERNATIVE DIFFERENTIATION METHODS

TODO: že existují nějaký další metody, na něco se hodí možná

A. Finite difference method

TODO: co je FD (finite difference) metoda, jak funguje, plusy a minusy

B. Symbolic method

TODO: co je symbolická metoda, jak funguje, plusy a minusy

C. Comparison

TODO: tabulka s porovnáním + vysvětlení proč

IV. AUTOMATIC DIFFERENTIATION

TODO: že existují dvě varianty, začneme tím, co mají společného

A. Core principles

TODO: proč to funguje, teorie za tím

TODO: chain rule, výpočetní graf, potřeba mít známé funkce a jejich derivace

TODO: “jediný” rozdíl mezi FM/BM je pořadí derivací v chain rule - dá se vysvětlit i jako pořadí násobení jakobiánů

B. Forward mode: The tangent linear mode

TODO: důležitý je “přenášení” derivací spolu s hodnotou ve výpočtu

TODO: duální čísla $\epsilon^2 = 0$ jako další matematický pohled na věc

TODO: vhodné pro málo vstupů, hodně výstupů - pro každý vstup třeba další průchod

C. Backward mode: The adjoint mode

TODO: potřeba dva průchody - první k vytvoření výpočetního grafu a zpětný k tomu, abychom viděli jak který vstup kontribuuje k výsledné hodnotě (kterou si zvolíme) -> víc paměti. taky VJP (vector jacobian product) - a na tom ukázat, že je vhodné pro mnoho vstupů a málo výstupů (vs forward mode)

D. Computational complexity

TODO: časová a paměťová komplexita, porovnání s ostatními metodami

V. MINIMAL IMPLEMENTATION

TODO: jak dát do kódu, python - fundamentální rozdíl mezi FMode a BM

A. Forward mode

TODO: + vysvětlení pomocí duálních čísel, přetížení standardních algebraických operací tak, aby se přenášela i derivace

B. Backward mode

TODO: + vysvětlení jak se používá computational graf k zpětnému chodu

VI. USING EXISTING AUTODIF LIBRARIES IN PYTHON

TODO: předpokládám python + pytorch, tensorFlow

VII. CONCLUSION

TODO: jak cca funguje autodif, k čemu je dobrá (výhody oproti FD a symbolic), důležitost pro reálné aplikace, limitace

VIII. FURTHER READING

TODO: list mých zdrojů s lepším popisem, pro ty, které by zajímalo víc

[1], [2], [3]

REFERENCES

- [1] Wikipedia contributors, "Automatic differentiation — Wikipedia, The Free Encyclopedia." [Online]. Available: https://en.wikipedia.org/w/index.php?title=Automatic_differentiation&oldid=1339245165
- [2] Andrew Holmes, "What's Automatic Differentiation?." [Online]. Available: <https://huggingface.co/blog/andmholm/what-is-automatic-differentiation>
- [3] Carnegie Mellon University, "Deep Learning Systems - Algorithms and Implementation." [Online]. Available: <https://dlsyscourse.org/>